Introduction to Web Technology

Web Technology — Module 1: Preliminaries (The Internet & The Web)

Instructor - Dr Suchetana Chakraborty

Formatted by: **Heisenberg** 😎

Index

- Course Roadmap — nine modules, one very long semester, zero shortcuts
- Resources & Evaluation Policy — the books you'll pretend to read and the marks you'll actually chase
- Learning Objectives — what you're contractually obligated to understand by the end of this module
- What Happens When a Webpage Loads — nine steps between "Enter" and "there's the cat picture"
- The Web Development Ecosystem — every acronym you'll eventually Google at 2 a.m.
- What Is the Internet? — spoiler: it's not a cloud, it's wires all the way down
- Internet Structure — Building the Network of Networks — how a scaling nightmare turned into ISPs, tiers, and peace treaties
- Host-to-Host Communication over the Internet — the five-layer relay race your data runs every single time
- The Web: Client, Server, and How They Talk — browsers ask nicely, servers (usually) answer
- Basic Architecture of the Web — A Worked Example — one page load, three servers, zero magic
- Process-to-Process Communication — sockets, the "doors" your data politely knocks on

---

Course Roadmap

This course covers nine modules across the full arc of web technology:

1. **Preliminaries** — Introduction to the Internet and World Wide Web, evolution of web technologies, client-server architecture, web browsers and web servers, URLs, DNS, HTTP and HTTPS protocols, static vs dynamic websites, overview of web application workflow.
2. **Webpage Design using HTML and CSS** — Structure of HTML documents, semantic HTML, headings, paragraphs, lists, tables, forms, multimedia elements, introduction to CSS, selectors and styling, colors, fonts, layouts, box model, responsive web design concepts, Flexbox and Grid basics.
3. **Client-Side Programming** — Introduction to JavaScript, variables and data types, operators and control statements, functions and events, DOM manipulation, form validation, interactive webpage development, basic asynchronous concepts.
4. **Web Application Architecture** — Introduction to MVC architecture, presentation layer (View), models and controllers, separation of concerns, overview of frontend and backend interaction, workflow of modern web applications.
5. **Databases and Backend Systems** — Introduction to databases, relational databases and SQL basics, CRUD operations, introduction to backend systems, web servers, server-side processing concepts, sessions and cookies, authentication basics.
6. **APIs and REST APIs** — Introduction to APIs, REST architecture, HTTP methods (GET, POST, PUT, DELETE), JSON format, request-response mechanism, API integration in web applications, use cases of public APIs.
7. **Frontend Development** — Introduction to modern frontend development, component-based interfaces, state and event handling concepts, overview of frontend frameworks and single-page applications.
8. **Web Application Security** — Common web vulnerabilities, SQL injection, cross-site scripting (XSS), authentication and authorization, HTTPS and secure communication, secure coding practices.
9. **Testing and Deployment** — Introduction to testing of web applications, functional testing, debugging techniques, browser developer tools, hosting and deployment basics, overview of cloud deployment, HTML5 and modern web technologies.

The web is fundamentally built on a **client-server paradigm**: the client is the browser side (Module 3 teaches you to program it), while the server side is where the data lives and the backend logic runs (Module 5). Module 4 sits between the two, introducing the **MVC (Model-View-Controller)** architecture — the standard set of layers used to organize how frontend and backend interact. Module 6 then covers how the client and server actually *talk* to each other in a structured way, using REST APIs. By Module 9, the goal is that the theoretical and programming aspects covered throughout the course converge into the practical skill of building and deploying your own end-to-end web application.

Lab Component

Alongside the theory, there are dedicated hands-on lab assignments covering:

- Creating webpages using HTML
- Styling webpages using CSS
- Developing interactive webpages using JavaScript
- Form validation and DOM manipulation
- Database connectivity demonstrations
- API integration using public APIs
- Debugging and testing webpages
- Deploying simple web applications

Each lab topic comes with its own assignment, and these are distinctly designed to build a specific, cumulative skill set for web development.

> **Note for Review:** Plagiarism in programming assignments is strictly prohibited and will be penalized. Since this course blends theory with hands-on skill-building, the assignments are meant to be a genuine learning checkpoint — not just a box to tick.

Resources & Evaluation Policy

Textbook: *Fundamentals of Web Development* (3rd Edition) by Randy Connolly and Ricardo Hoar — the standard reference for this course.

Reference Books:

- *HTML & CSS: Design and Build Websites* — Jon Duckett
- *JavaScript & jQuery: Interactive Front-End Web Development* — Jon Duckett

These are recommended pointers to follow alongside the course; more references will be shared as the course progresses.

Evaluation Policy:

Component	Weightage
Major Examination	60%
Lab Exam	20%
Quizzes (video + module quizzes)	20%
Programming Assignments	10%

This is a tentative allocation — the exact split within lab exam, quizzes, and assignments may be adjusted slightly (e.g., the lab-related components may be redistributed as one final lab exam plus quizzes, comprising roughly 25%, with programming assignments at 10%). Every lab topic covered will have a corresponding assignment.

Learning Objectives

By the end of this module, you should be able to:

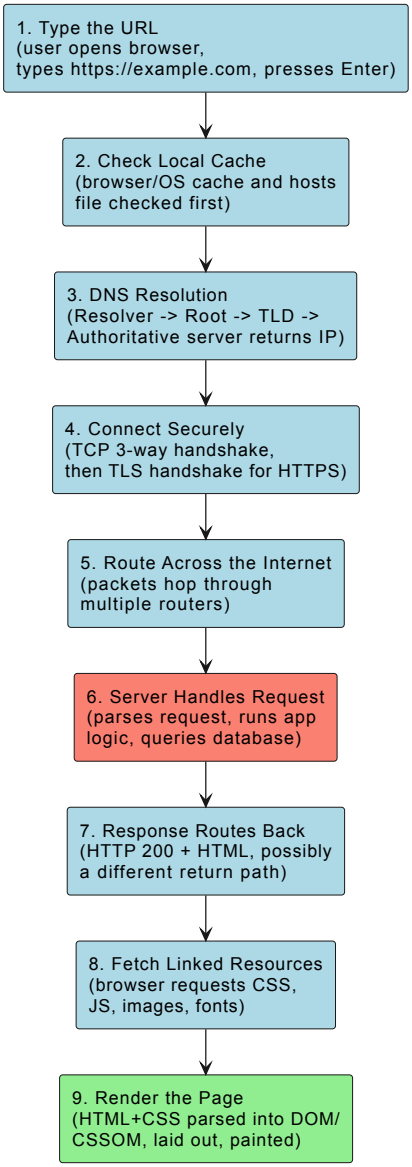
Objective	What it means
The Internet & WWW	Explain how the Internet and the World Wide Web relate to each other
Evolution of the Web	Describe how web technology evolved from static pages to cloud-native apps
Client–Server Model	Explain the client–server model and the roles of browsers and servers
Request Flow	Trace what happens — DNS lookup, HTTP request, response — when a URL loads

This module builds the architecture of the "global network" that has become a **content platform**. It traces the evolution of the web from Web 1.0 toward the cloud-native Web 3.0 (and beyond), explains the client-server architecture (how browsers and servers communicate), and touches on URLs, DNS, HTTP protocols, web caching, cookies, and finally the distinction between static and dynamic sites.

The bigger picture: understanding *how the technology evolved* helps explain *why* new opportunities keep opening up for the kinds of applications we build today — and the client-server model is the key paradigm underlying almost all of it.

What Happens When a Webpage Loads

You've probably noticed that some pages load almost instantly, while others render in bits and pieces — text first, then images, then styling snapping into place. This happens because **the content your browser fetches is never local to your system** — it has to travel, often across multiple networks, before it reaches you.



Walking through the flow:

1. **Type the URL** — You open a browser (Chrome, Brave, Edge — pick your poison) and type an address, e.g., the CSE department page at IIT Jodhpur, then press Enter.
2. **Check Local Cache** — The browser first checks whether it already has this page cached. Caching exists purely for performance: if you've visited the page recently, there's no need to burn bandwidth fetching it again.
3. **DNS Resolution** — If it's not cached, the browser needs to find *where* the content actually lives. DNS (Domain Name System) converts a human-friendly name like `iitj.ac.in` into the corresponding IP address, because the routers that carry your request only understand IP addresses, not names.
4. **Connect Securely** — Security protocols establish the end-to-end connection (the TCP handshake, followed by TLS for HTTPS).
5. **Route Across the Internet** — The request packets hop through multiple routers to reach the destination server.
6. **Server Handles Request** — The server is rarely dedicated to just one page; it often hosts many websites and services, and may talk to other backend servers before responding — using a protocol that is part of HTTP.
7. **Response Routes Back** — The response may take a different path back than the request took, but it follows the same routing principles across multiple networks until it reaches your local network.
8. **Fetch Linked Resources** — The browser requests any additional resources the page references (CSS, JS, images, fonts).
9. **Render the Page** — Finally, the content is rendered in your browser, and you see what you asked for.

All of this typically happens within a split second on a robust connection — but on a weaker one, you'll notice the page loading in visible stages.

The Web Development Ecosystem

Since this is an introductory course, it's worth seeing the bigger picture of *where* these skills lead.

Technology Stack	Examples
Frontend	HTML, CSS, JavaScript, React / Angular / Vue, responsive design
Backend	Node.js, Python, PHP, Java, .NET, Ruby (on Rails)
Databases	MySQL, PostgreSQL, MongoDB, Redis, data modeling
APIs & Integration	REST, GraphQL, Webhooks, 3rd-party APIs
DevOps & Hosting	Git, CI/CD, Docker, AWS/Azure/GCP, CDN, load balancing
Security & Network	HTTPS/TLS, Auth, DNS, firewalls, OWASP practices

Roughly speaking, **frontend is mostly a design job**, while **backend is mostly a dev job** — backend programming handles server-side activities and business logic.

This stack maps onto a set of career roles:

Role	Focus
UX/UI Designer	Wireframes, prototyping, design systems, user research
Frontend Developer	HTML/CSS/JS, component frameworks, accessibility
Backend Developer	APIs, business logic, database design, server performance
Full-Stack Developer	Frontend + backend, end-to-end features, systems thinking
DevOps Engineer	CI/CD pipelines, cloud infrastructure, monitoring, scaling
QA & Security	Testing, debugging, vulnerability checks, secure coding review

Not every one of these technologies will be covered in this course — but the goal is to build a solid grip on the *fundamentals*, so that picking up any specific language or framework later becomes easy. Languages are just different ways of implementing the same underlying rationale of how web applications are built.

What Is the Internet?

At the simplest level, when two devices — say a phone (X) and a laptop (Y) — run the same application (e.g., a WhatsApp Web chat) and communicate with each other, "the Internet" is the abstraction sitting between them: a cloud that just *works*.

But underneath that abstraction, there's a lot going on, and understanding it in some detail makes it much easier to talk about the rest of this course.

The Internet is, fundamentally, a network of networks.

It's made up of many different underlying network technologies — cellular (4G/5G), Wi-Fi (home networks), Ethernet (enterprise networks), and datacenter networks in the cloud. All of this infrastructure exists to connect **hosts** (end devices): smartphones, laptops, connected vehicles, wearables, and so on.

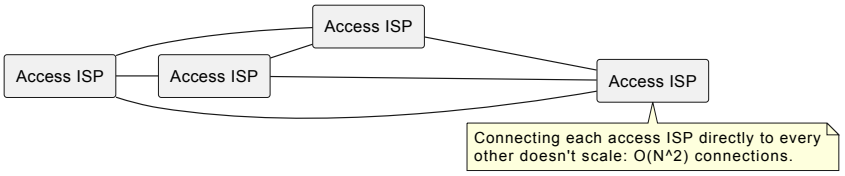
- **Network edge** — the hosts/devices and access points at the boundary of the network.
- **Network core** — the servers and infrastructure ("network of networks") that provide the actual connectivity backbone.

To bridge the heterogeneity of all these different device and network types, everyone follows shared **protocols** — standardized rules for communication, such as HTTP, TCP, and IP. Protocols are what let a phone on 4G talk to a server on Ethernet without either side needing to know the details of the other's hardware.

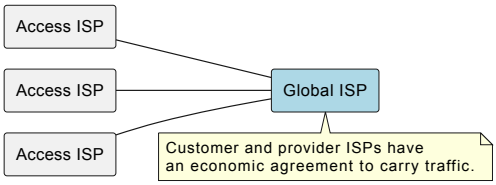
Internet Structure — Building the Network of Networks

Hosts connect to the Internet via **access ISPs** (Internet Service Providers) — the entities that build, maintain, and manage the actual physical infrastructure carrying packets between hosts.

The scaling problem: if every access ISP tried to connect directly to every other access ISP (a mesh/"click" structure), you'd need on the order of $O(N^2)$ connections — completely infeasible as the network grows, even though *any* host still needs to be able to reach *any* other host.

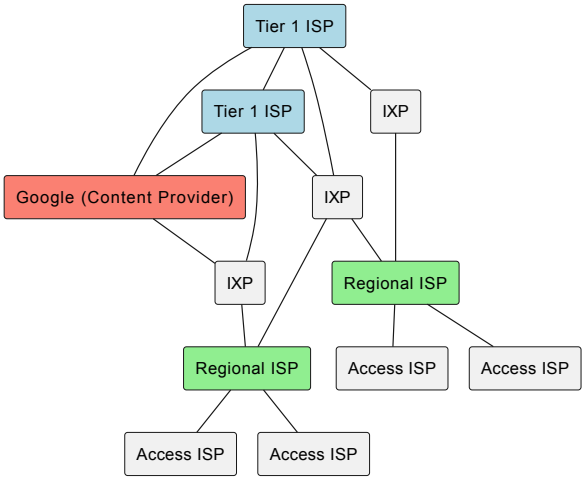


Attempt 1 — One global transit ISP: connect every access ISP to a single **global ISP**, under a customer–provider economic relationship (you pay the global ISP to carry your traffic to another access network). This solves the scaling problem, but a single global ISP isn't feasible either — coverage of the entire globe, business-model concerns, and scalability all break down.



Attempt 2 — Multiple competing ISPs: since one global ISP isn't feasible, multiple ISPs (ISP A, ISP B, ISP C, ...) each build out their own regional backbones, with access ISPs connecting to whichever provider serves them.

Attempt 3 — Peering and content providers: these regional ISPs then interconnect with each other at **Internet Exchange Points (IXPs)**, and via direct **peering links**. Meanwhile, large **content provider networks** (Google, Microsoft, Akamai, Facebook) build their *own* private networks to bring content physically closer to end users, often bypassing tier-1 and regional ISPs entirely.



The final, real-world structure is a hierarchy: at the "center" there is a small number of well-connected large networks —

- **Tier-1 commercial ISPs** (e.g., Level 3, Sprint, AT&T, NTT) providing national and international coverage.
- **Content provider networks** (e.g., Google, Facebook) — private networks connecting their own data centers to the Internet, often bypassing tier-1 and regional ISPs.

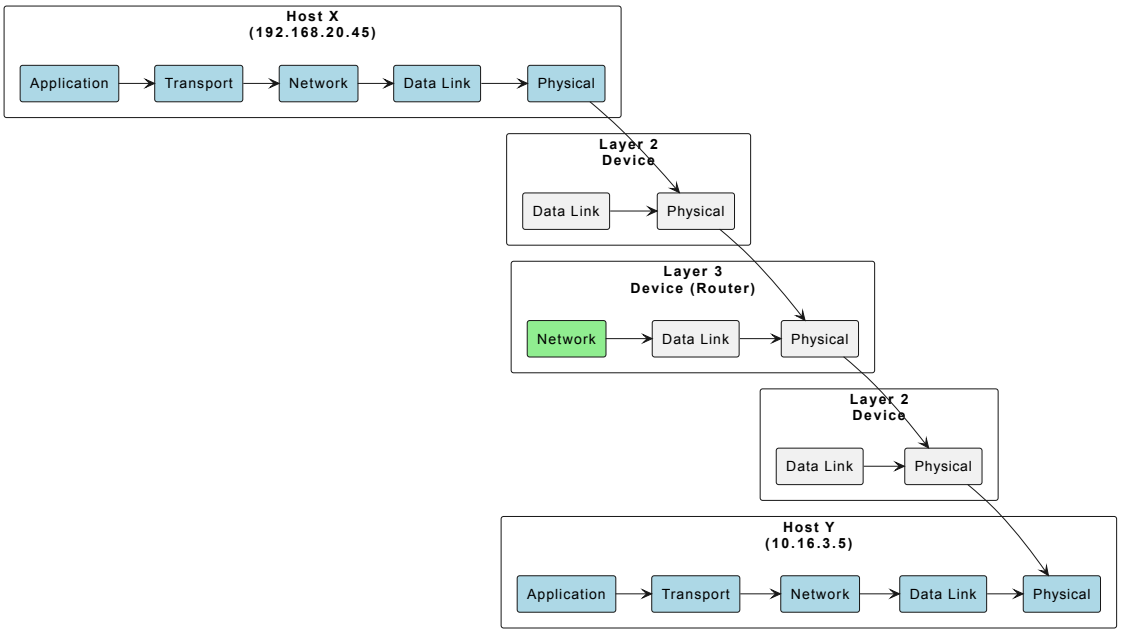
Below that sit **regional ISPs**, and below those, the **access ISPs** that individual hosts connect through. This entire structure is not centrally designed — it's an emergent, evolving system shaped by economics, national policy, and business relationships between providers (who peers with whom, who pays whom).

Worked example — packet path: Suppose host X is on one access network and host Y is on another, geographically distant one. X's device sends a packet over a wireless channel to its local **Wi-Fi access point**, which connects to a **layer 2 device** (a switch) managing local network functions — handling things like fading, multipath effects, and interference on the shared wireless channel. From there, the packet is forwarded up to a **layer 3 device** (a router), which makes the actual routing/forwarding decision — this is the essence of routing protocols. The packet then hops across multiple routers, eventually reaching a router in Y's access network, and finally Y's local base station/access point delivers it to Y's device.

> **Note for Review:** The distinction between a *layer 2 device* (switch) and a *layer 3 device* (router) is a favorite exam point. A switch only needs to operate up to the Data Link layer to forward frames within a local network; a router operates up to the Network layer because its job — routing and forwarding across different networks — genuinely requires network-layer (IP) information.

Host-to-Host Communication over the Internet

Now zoom into what happens *inside* each host when it sends or receives data, using the standard **TCP/IP protocol stack**:



Walking down and back up the stack:

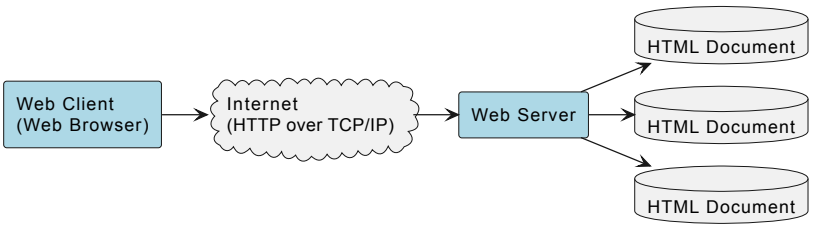
- **Application layer** — Suppose the same application process (say a web browser) runs on both X and Y; this is where application data is generated.
- **Transport layer** — The generated data is divided into chunks and handed to the transport layer, which establishes an end-to-end (virtual) connection — for example, a TCP connection acts like a secure virtual tunnel, even though physically the packets are just a stream of 0s and 1s traveling over wires or radio.
- **Network layer** — Handled by the host's NIC (network interface card); this is where routing/forwarding logic lives.
- **Data Link layer** — Manages how multiple hosts sharing the same channel (e.g., several devices on the same Wi-Fi) cooperate and compete for bandwidth — this is where flow control and error control happen. Protocols like ARP/RARP operate here.
- **Physical layer** — The actual transmission medium: radio channel for wireless, or coaxial/copper cable for Ethernet — the literal 0s and 1s moving through hardware.

A **layer 2 device** (like a switch) only needs to operate up through the Data Link layer, since its job is simply to connect hosts within a local segment — it has no need for network-layer services. A **layer 3 device** (a router) operates up through the Network layer, since its job is to perform routing and forwarding, connecting to many other routers to form the core of the network.

The packet's journey: **Application** → **Transport** → **Network** → **Data Link** → **Physical** on the sending host, across potentially many layer-2 and layer-3 devices (routers), and then back **up** through Physical → Data Link → Network → Transport → Application on the receiving host. At the transport layer on the receiving end, the packet must be matched to the correct **process** (since a host runs many processes simultaneously) before the application data is finally delivered.

The Web: Client, Server, and How They Talk

The **web** is a specific application built on top of the Internet's connectivity infrastructure, structured around a **client** and a **server** that may be geographically far apart.



- The **web client** is typically a web browser — the side that *initiates* requests.
- The **web server** hosts multiple documents. A **website** is a collection of interlinked pages you can navigate across.
- **HTTP** (HyperText Transfer Protocol) is an **application-layer protocol** that relies on the transport-layer protocol **TCP** and the network-layer protocol **IP**. Together, these three layers support the web: the client sends a *request* to the server, and the server sends back a *response* containing the requested resources, which the browser then renders.

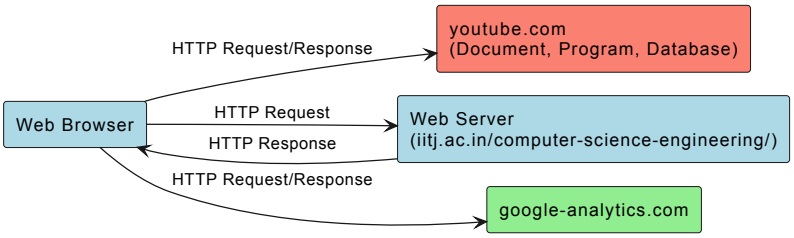
Basic Architecture of the Web — A Worked Example

Consider visiting the IIT Jodhpur CSE department page. Breaking apart the **URL** (Uniform Resource Locator):

```
https://iitj.ac.in/computer-science-engineering/
```

- `https` — the protocol (secure HTTP)
- `iitj.ac.in` — the **domain name** (address of the host server)
- `/computer-science-engineering/` — the **path**, locating a specific page hosted on that server

A URL therefore encodes two things: *which server to reach*, and *which specific page on that server to fetch*.



When this page loads, several things happen behind the scenes, each involving a separate client-server exchange:

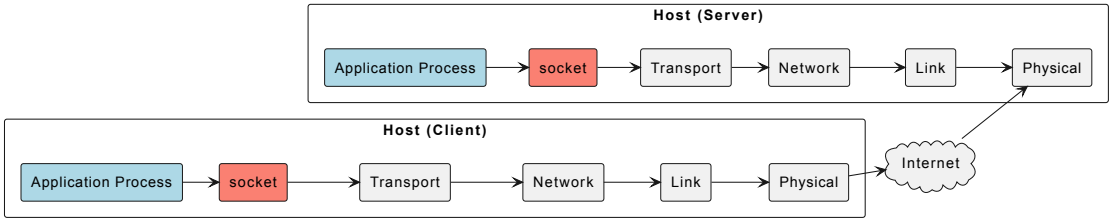
1. The browser sends a **GET request** (one of HTTP's request methods, used to *fetch* data) for the page — e.g., the index HTML — from the web server at `iitj.ac.in`.
2. Simultaneously, the page might send a request to **google-analytics.com**, a completely different server, to log/report visit rankings and analytics.
3. If the page embeds a **YouTube** video, clicking on it triggers a request to YouTube's servers, which may in turn query a database for the actual media/document, before sending it back.

In every one of these cases, the pattern is the same: **the client (browser) always initiates the request; the server always responds with data.** The server's response — a **static or dynamic web page**, typically built with HTML, CSS, and JavaScript — is then rendered by the browser for the user to see.

> **Note for Review:** "Static vs. dynamic" is a key distinction this module sets up for later: a *static* page is the same HTML sent to every visitor, while a *dynamic* page is generated (often per-request, often from a database) on the server before being sent. We'll unpack this further along with HTTP methods in upcoming sessions.

Process-to-Process Communication

Zooming in one more level: communication over the Internet isn't really host-to-host — it's **process-to-process**. A process sends and receives messages through its **socket**.



A helpful analogy: **a socket is like a door.**

- The sending process shoves its message out through the door (the socket).
- It then relies entirely on the transport infrastructure *on the other side of the door* to carry the message to the socket at the receiving process.
- Every communication involves **two sockets** — one on each side.

Two different parties are responsible for two different sides of this picture:

- The **application process and its socket** are controlled by the **app developer** — this is the code you actually write.
- Everything below the socket (transport, network, link, physical) is controlled by the **operating system**.

> **Note for Review:** This client/OS split is worth remembering — as an application developer, your entire job is to correctly open, write to, and read from a socket. You never have to personally implement TCP's handshake or IP's routing; the OS and network stack handle all of that beneath the socket abstraction.

This concludes the Preliminaries module's coverage of Internet fundamentals. The next sessions continue into DNS in detail, followed by HTTP/HTTPS, web caching and cookies, and the static-vs-dynamic web distinction.